Passermark – Command line

Manual for `passermark-cli` with case studies · Passermark 1.18.0

The command line runs Passermark's heavy functions without a GUI: in scripts, for whole folders at once, or later automatically in the background. Everything described here does the same as the corresponding dialogs in the program.

1 · Invocation

```
passermark-cli <job> <input.pdf> <output.pdf> [options]
```

System	What the command is called
Linux (installed with install.sh)	`passermark-cli …`
Linux (AppImage)	`./Passermark-*-x86_64.AppImage --cli …`
Windows	`"C:\Program Files\Passermark\passermark-cli.exe" …`
Windows (path option ticked during installation)	`passermark-cli …`

Windows: since 1.7 the setup has the option “Make passermark-cli available in every command prompt”. Then `passermark-cli` is enough – in a **newly opened** Command Prompt or PowerShell window. Without the option, always give the full path in quotation marks.

The input file is never modified. The output is only written when everything is finished – after an error or cancellation no half-written file is left behind. Input and output must not be the same file.

The ten jobs

Job	What it does
`cutcontour`	Cut line for a cutting plotter (spot colour CutContour), optionally with bleed
`separate`	Detect motifs, each motif as its own page
`manip`	CMYK conversion with an ICC profile and/or cropping to a format
`repair`	Repair, optimise for printing, shrink, PDF/A, password
`preflight_fix`	Flatten layers, embed fonts, outline text, flatten transparency
`impose`	Imposition as in the print dialog: booklets and signatures, step & repeat, N-up, posters
`vdp`	Variable data: numbers, texts from CSV, QR codes, Code 128, EAN-13 – every page the next record
`split`	Split pages: halves or a grid, e.g. a booklet exported as spreads into single pages
`shrink`	Reduce file size: lossless measures can be switched individually, optional image downsampling
`press`	Output for the print shop: convert as recommended, always embed the output intent, optionally a package with profile and report

Options

Option	Meaning
`--set name=value`	Set one setting; can be repeated. Numbers, `true`/`false` and lists are recognised, everything else is text. Nested with a dot: `--set detect.tolerance=40`
`--preset file.json`	Load settings from a file (see section 3). `--set` overrides individual values from it
`--preset "Name"`	Use a preset saved in the program (see section 3)
`--pages 1,3-5`	Process only these pages (where applicable)

Option	Meaning
`--quiet`	No progress display
`--json` / `--json-progress`	Progress and result as JSON lines – for other programs
`list`	Show all jobs: `passermark-cli list`
`settings <job>`	All settings of a job with their defaults as JSON – the template for your own presets
`presets [job]`	List the presets saved in the program
`reprint protokoll.json`	print a job from the print log again, exactly the same (section 6)
`plan plans.pdf …`	Plan printing on rolls: roll choice, run length, printing (section 6)
`rolls PRINTER`	the rolls loaded in the printer (from the device or entered by hand)
`tidy datei.json`	cut a preset or workflow down to the values that differ from the default (`--full` = preset with all values, `--print` = only show)
`beispiele [folder]`	Copy the variable-data examples to the user folder (default: `Passermark-Beispiele`) and register them as presets “Beispiel 1 …”
`datenarten`	Data kinds for tables (business card, Wi-Fi, e-mail, Code 128, EAN-13 …) with their columns
`datenvorlage <kind> [file.csv]`	Empty table with the matching columns and one example row
`datencheck <file.csv> [kind]`	Check a table – reports row and error; return code 1 on problems
`run`, `check`, `workflows`	Run, check, list workflows (section 5)
`printers`, `caps`	Printers and their options (section 6)
`inspect`	Examine a document (section 7)
`version`, `schema`	for other programs (section 8)
`serve`	Watch drop folders (section 9)

Return codes and cancelling

`0` = done · 1 = error (e.g. file missing, no motif found) · 2 = wrong invocation · `130` = cancelled. **Ctrl+C** cancels cleanly, without an output file. In scripts you can react to errors this way (section 4).

2 · The jobs with case studies

cutcontour – cut line for the plotter

Setting	Default	Meaning
`shape`	`contour`	Shape: `contour` (follows the motif), `rect`, `rounded`, `circle`, `oval`, `hexagon`, `octagon`, `heart`, `star`, `shield`, `arch`
`offset_mm`	0	Distance of the line from the motif: + outwards (white border), − inwards
`smooth_mm`	1,5	Smoothing of the contour (deliberately does not follow the motif exactly)
`bleed`	`true`	Create bleed (colour runs beyond the cut line)
`bleed_mm`	2	Width of the bleed
`inner`	`false`	Also cut inner contours (holes in the motif)
`width_mm` / `height_mm`	0	Basic shape: size of the cut line in mm (0 = from the motif)
`shift_x_mm` / `shift_y_mm`	0	Basic shape: offset from the motif centre (+ right / + up)
`corner_mm`	3	Corner radius for `rounded`
`single_shape`	`true`	Basic shape: one shape around the whole motif; `false` = one per object (sticker sheet)
`per_object`	`false`	Each object on its own page
`margin_mm`	6	Page margin for `per_object`
`spot`	`CutContour`	Name of the spot colour – differs depending on the RIP software
`stroke_pt`	0,25	Line width
`dpi`	200	Accuracy; 300 easily takes twice as long
`detect.*`		Motif detection, see `separate`

Sticker sheet with contour and 2 mm bleed

Every sticker gets its contour, the colour runs 2 mm beyond the line – no white flash when cutting.

```
passermark-cli cutcontour bogen.pdf bogen-cut.pdf --set bleed_mm=2
```

Logo with an 80 × 50 mm rectangle, without bleed

Fixed size of the cut line, centred on the whole logo – even if the logo consists of several parts.

```
passermark-cli cutcontour logo.pdf logo-cut.pdf \
    --set shape=rect --set width_mm=80 --set height_mm=50 --set bleed=false
```

Round stickers Ø 60 mm on a sheet

Every sticker individually (`single_shape=false`), each circle exactly 60 mm.

```
passermark-cli cutcontour bogen.pdf rund.pdf \
    --set shape=circle --set width_mm=60 --set height_mm=60 --set single_shape=false
```

3 mm white border around the motif

The contour lies 3 mm outside; without bleed the border stays white.

```
passermark-cli cutcontour logo.pdf logo-rand.pdf --set offset_mm=3 --set bleed=false
```

Every sticker on its own page

Practical for individual jobs or when the plotter cuts page by page.

```
passermark-cli cutcontour bogen.pdf einzeln.pdf --set per_object=true --set margin_mm=5
```

Different spot colour for the RIP software

Some programs expect a different name than “CutContour”, e.g. “Thru-cut”.

```
passermark-cli cutcontour bogen.pdf bogen-cut.pdf --set spot=Thru-cut
```

Only specific pages

Process pages 1 and 3 to 5.

```
passermark-cli cutcontour mappe.pdf mappe-cut.pdf --pages 1,3-5
```

separate – motifs as individual pages

Setting	Default	Meaning
`margin_mm`	0	Margin around each motif (negative = inwards)
`detect.mode`	`auto`	`auto`, `transparent` (cut out) or `color` (solid-colour background)
`detect.tolerance`	28	How much the motif must differ from the background (0–255); higher = less sensitive to spots
`detect.min_size_mm`	5	Smaller parts count as dust
`detect.gap_mm`	1	Parts lying closer together belong to one motif

Split a scan with several motifs

Each motif becomes a page, with a 3 mm margin.

```
passermark-cli separate scan.pdf motive.pdf --set margin_mm=3
```

Blotchy background (e.g. recycled paper)

Higher tolerance and larger minimum size – spots are no longer detected as motifs.

```
passermark-cli separate scan.pdf motive.pdf \
    --set detect.tolerance=40 --set detect.min_size_mm=10
```

manip – CMYK conversion and cropping

Setting	Default	Meaning
`crop`	`false`	Bring to a format (excess cut equally on both sides)
`crop_scale`	`true`	First scale until the format is completely filled (A4 → A6 = 50 %), then trim the excess of the other edge; `false` = only trim (files with bleed)
`crop_inset_mm`	0	Trim this much all around beforehand (white borders of the original)
`crop_size`	A4	A0–A7, B4, B5, C4–C6, DL, SRA4, SRA3, SRA2, A3+, Letter, Legal, Tabloid, “business card 85×55”, “business card 90×50”, “square 210”, “square 148” or `custom`
`crop_w_mm` / `crop_h_mm`	210 / 297	Dimensions for `custom`
`crop_follow`	`true`	Follow the page's portrait/landscape orientation
`cmyk`	`false`	Convert to CMYK (needs Ghostscript)
`target`		Path to the CMYK destination profile (.icc), e.g. ISOcoated_v2_eci.icc
`cmyk_mode`	`rgb_only`	`rgb_only` (existing CMYK stays), `all` (also convert CMYK, source profile in `source`), `gray`
`intent`	`relative`	`perceptual`, `relative`, `saturation`, `absolute`
`gray_to_k`	`true`	Print grey/black with K only

Bring all pages to A5

A4 is scaled down to 70.7 %; a narrower motif fills the width, an equal amount is trimmed top and bottom.

```
passermark-cli manip flyer.pdf flyer-a5.pdf --set crop=true --set crop_size=A5
```

File with 3 mm bleed to trim size (trim only)

```
passermark-cli manip flyer-anschnitt.pdf flyer-a4.pdf \
    --set crop=true --set crop_size=A4 --set crop_scale=false
```

Custom format 100 × 150 mm

```
passermark-cli manip foto.pdf foto-10x15.pdf \
    --set crop=true --set crop_size=custom --set crop_w_mm=100 --set crop_h_mm=150
```

Print data to CMYK (ISO Coated v2)

The profile is available free of charge from the ECI (eci.org). Adapt the path to where you store it.

```
passermark-cli manip druck.pdf druck-cmyk.pdf \
    --set cmyk=true --set target=/pfad/ISOcoated_v2_eci.icc
```

press – Output for the print shop

Like **Colour → Output for print shop** in the program: checks the file against the printing condition, converts what is needed, always embeds the profile as output intent and checks the result. If the file contains RGB, `rgb` must be set – the command line does not ask, it stops.

Setting	Default	Meaning
`target`		path to the CMYK profile of the printing condition (.icc), e.g. PSOcoated_v3.icc – required
`cmyk_mode`	`auto`	`auto` (as the colour check recommends), `rgb_only`, `all` (separate everything again), `none` (output intent only)
`rgb`	`ask`	`convert` (convert to the target profile), `keep` (pass RGB on with profile, RGB without profile becomes sRGB), `ask` = error if there is RGB
`source`		source profile for existing CMYK with `all` (empty = from the output intent of the file)
`intent`	`relative`	`perceptual`, `relative`, `saturation`, `absolute`
`tac_limit`	0	guide value for ink coverage in % (0 = usual value of the printing condition)
`package`	`false`	additionally `<output>_Druckerei.zip`: PDF, profiles used, Bericht.txt (report)

Flyer for the offset print shop (coated paper)

RGB images are converted, the profile is embedded as output intent, plus the package for queries. The profiles of the standard printing conditions are fetched in the program under Colour → Get standard profiles.

```
passermark-cli press flyer.pdf flyer-druck.pdf \
    --set target=/pfad/PSOcoated_v3.icc --set rgb=convert --set package=true
```

repair – repair, optimise, PDF/A, password

Setting	Default	Meaning
`mode`	`print`	`repair` (repair only), `print` (for sharing and printing, embed fonts), `screen` (shrink for e-mail, 150 dpi), `pdfa` (PDF/A-2b, archive/authorities)
`password`		Password of the input file if it is protected

Setting	Default	Meaning
new_password		New password for the output (empty = none); not with pdfa – PDF/A forbids encryption, the password is then omitted
allow_print	true	With a password: still allow printing

Prepare a broken PDF for printing

```
passermark-cli repair kaputt.pdf ok.pdf --set mode=print
```

Shrink for e-mail

```
passermark-cli repair katalog.pdf katalog-klein.pdf --set mode=screen
```

Archive as PDF/A

For archives, authorities and e-invoice attachments.

```
passermark-cli repair vertrag.pdf vertrag-a.pdf --set mode=pdfa
```

Protect with a password

Opening only with a password, printing stays allowed.

```
passermark-cli repair angebot.pdf angebot-geschuetzt.pdf --set new_password=geheim
```

preflight_fix – make problem PDFs print-ready

Setting	Default	Meaning
fix	flatten_layers	flatten_layers (fix layers – exactly what is visible is printed), embed_fonts, outline_text (text to paths), flatten_transparency, flatten_softmask (transparent gradients to opaque images – against black areas on the RIP, no Ghostscript needed)
font_map	{}	With embed_fonts: substitute fonts {font name: path}
dpi	300	With flatten_transparency and flatten_softmask: resolution of the rasterised areas

Plan with layers: print only the visible

```
passermark-cli preflight_fix plan.pdf plan-fest.pdf --set fix=flatten_layers
```

Old RIP: flatten transparency

```
passermark-cli preflight_fix plakat.pdf plakat-flach.pdf \
    --set fix=flatten_transparency --set dpi=300
```

embed_fonts, outline_text, flatten_transparency, repair and the CMYK conversion need Ghostscript. The Windows setup and AppImage include it; with the Linux installation install.sh installs it.

shrink – reduce file size

Every measure is a separate switch – if one causes problems, just turn it off with --set name=false. No Ghostscript needed. If the result is not smaller, the file is taken over unchanged.

Setting	Default	Meaning
rewrite	true	rewrite: old saved versions removed, compressed object streams, stronger ZIP
unreferenced	true	remove unused resources
dedupe	true	merge images, fonts, ICC profiles stored twice

Setting	Default	Meaning
`private`	`true`	remove Illustrator/Photoshop editing data (PieceInfo)
`thumbs`	`true`	remove embedded page thumbnails
`pack`	`true`	compress images losslessly (ZIP with predictor), remove ZIP around JPEG – only if smaller
`bits16`	`false`	16-bit images to 8 bit
`attachments`	`false`	remove embedded files (attachments)
`hidden`	`false`	remove content of hidden layers
`downsample`	`false`	downsample images above `target_dpi` × `threshold` (measured at the largest placement) to `target_dpi`
`target_dpi`	300	target resolution for downsampling
`threshold`	1.5	only above target × threshold (1.5 at 300 dpi = from 450 dpi)
`photos_jpeg`	`false`	save photos stored as ZIP as JPEG (real photos only, only if clearly smaller)
`jpeg_quality`	92	JPEG quality for downsampling and for `photos_jpeg`

Reduce losslessly only

The printed result stays pixel-identical.

```
passermark-cli shrink flyer.pdf flyer-klein.pdf
```

Poster: images to 150 dpi

Keep the editing data.

```
passermark-cli shrink plakat.pdf plakat-klein.pdf \
    --set downsample=true --set target_dpi=150 --set private=false
```

impose – impose booklets, signatures and step & repeat

The same as the print dialog, but as a PDF file: every output page is a press sheet. The sheet format is set in `sheet`. `passermark-cli settings impose` shows all print dialog settings; the most important:

Setting	Default	Meaning
`sheet`	`A4`	Sheet: A0–A6, A3+, SRA3, B4–B6, LETTER, LEGAL, TABLOID or `custom` with `sheet_w_mm`/`sheet_h_mm`
`handling`	`size`	`booklet`, `multiple` (several per sheet), `poster`
`booklet_kind`	`saddle`	`saddle` = saddle stitch (all sheets nested), `grouped` = signatures of several sheets, `stack` = single sheets (perfect binding)
`booklet_per_sig`	4	With `grouped`: sheets per signature (4 sheets = 16 pages)
`booklet_creep_mm`	0	Creep per sheet, roughly the paper thickness (0.1 for 80 g/m²)
`booklet_blanks`	`end`	Blank pages `end` or `before_back` (the back cover stays the last page)
`booklet_blank_text`	–	Notice text on added blank pages
`booklet_fold_marks`	`false`	Fold marks
`booklet_reg_marks`	`false`	Registration marks
`booklet_collation_marks`	`false`	Collation marks on the spine of each signature
`booklet_gutter_mm`	0	Gutter (distance to the fold)
`step_repeat`	`false`	Step & repeat: one page several times per sheet; `sr_by=long`/`short` + `sr_mm` for the repeat size
`bleed_mm`, `crop_marks`	0, `false`	Bleed (mirrored) and crop marks

A5 booklet on A4 (saddle stitch)

Blank pages before the back cover, fold marks.

```
passermark-cli impose heft.pdf heft-bogen.pdf --set handling=booklet --set sheet=A4 \
    --set booklet_blanks=before_back --set booklet_fold_marks=true
```

Thick book in 16-page signatures

4 sheets per signature, creep 0.1 mm, collation marks.

```
passermark-cli impose buch.pdf buch-bogen.pdf --set handling=booklet --set sheet=A3 \
    --set booklet_kind=grouped --set booklet_per_sig=4 --set booklet_creep_mm=0.1 \
    --set booklet_collation_marks=true
```

A4 flyer as A6 8× on SRA3

Long edge 148 mm, 2 mm bleed, crop marks.

```
passermark-cli impose flyer.pdf flyer-sra3.pdf --pages 1 --set sheet=SRA3 \
    --set step_repeat=true --set sr_by=long --set sr_mm=148 \
    --set bleed_mm=2 --set crop_marks=true
```

Print presets from the program apply here directly – including the paper size: set them in the print dialog, **Save…** at the top, then `--preset "Name"`. The print order of the sheets is designed for duplex printing over the **short edge**. If the PDF has a trim size with bleed (TrimBox, e.g. from InDesign with “use bleed”), the trim size is placed and the bleed comes from the document – `sr_mm` then refers to the trim size.

vdp – variable data: numbers, names, QR and bar codes

Places fields on the template – every page gets the next record. Records come from a CSV file (first line = column names; the separator ; , or tab is detected) or from pure numbering (`count`). Contents are templates: `{{Spalte}}`, `{{nr}}` (formatted number), `{{i}}` (running from 1). Everything stays vector.

The easiest way to create the fields is in the program (Variable data workspace, Ctrl+Shift+D, place the fields with the mouse, preview per record) and save a preset. The command line then takes the preset and changes only what differs per job:

Setting	Default	Meaning
`csv_path`	–	CSV file; empty = numbering only
`count`	1	Without CSV: this many copies
`records`	–	Only these records, e.g. 1-50
`numbering.start` / `.step` / `.digits`	1 / 1 / 0	Start, step, zero-padded to this many digits
`numbering.prefix` / `.suffix`	–	Text before/after the number
`numbering.check`	`none`	`luhn`, `ean`, `mod11` – append a check digit
`numbering.continue_key`	–	Counter name: the next job continues counting there
`reverse`	`false`	Backwards (tear-off pad: top sheet has the highest number)
`pages_per_record`	1	With each: this many pages per record – 2 = front and back
`fields[].pages`	–	Field only on these template pages: `1,3-4`, `ungerade`, `gerade`, `ungerade 1-50`
`order`	each	each = every page the next record (one-page template: one page per record), `record` = all pages per record, page = like record, page by page
`log_path`	–	Code log as CSV (output page, record, values)
`placeholders`	`false`	Take {{…}} in the PDF as the field position, remove the placeholder text
`fields[].qr_type`	`text`	QR field: `vcard`, `wifi`, `email`, `url`, `phone`, `sms`, `event`, `geo` – content from the table columns (in the preset; in the program “QR code type”)
`fields[].qr_map`	`{}`	Mapping of item → CSV column, e.g. `{"Position": "Title"}`; `-` = leave empty; items not mentioned are recognised automatically (also Outlook/Google exports)

Ready-made examples with template, CSV and presets (QR code, Code 128, EAN-13) are fetched by `passermark-cli beispiele`; explained in detail in the “Variable data” manual (Help → Variable data – Manual).

500 numbered tickets

Create the "Tickets" preset in the program first (field with {{nr}}).

```
passermark-cli vdp ticket.pdf tickets.pdf --preset "Tickets" --set count=500
```

Name badges from a guest list

Fields {{Name}} and a QR code with {{Name}}; plus a log file.

```
passermark-cli vdp schild.pdf schilder.pdf --preset "Namensschilder" \
    --set csv_path=gaeste.csv --set log_path=codes.csv
```

Business cards with a QR code from a contact list

Create an empty table, fill it in Excel, check, generate. The preset has a QR field of type “business card” (qr_type vcard).

```
passermark-cli datenvorlage vcard kontakte.csv
passermark-cli datencheck kontakte.csv
passermark-cli vdp karte.pdf karten.pdf --preset "Visitenkarte" --set csv_path=kontakte.csv
```

Every card with its own code 8× on SRA3, cut and stack

First the data, then imposition – the next page per repeat, the stacks stay in order.

```
passermark-cli impose tickets.pdf tickets-sra3.pdf --set sheet=SRA3 \
    --set step_repeat=true --set sr_sequence=true --set sr_stack=stack \
    --set crop_marks=true
```

split – split pages (halves, grid, spreads)

Splits pages into single pages – lossless, the content is not modified. If the page has a trim size, it is split within it and the bleed stays outside. Rotated pages are split the way you see them. In the program: select pages, right-click → Split → halve; or Pages → Split pages … / Split spreads.

Setting	Default	Meaning
`mode`	`halves_v`	`halves_v` = left | right, `halves_h` = top / bottom, `grid` = grid
`cols` / `rows`	2 / 2	Only with `grid`: columns × rows
`only_spreads`	`false`	Only split pages that are twice as wide (or grid-times as large) as the others
`rtl`	`false`	Order right to left (right binding)
`use_trim`	`true`	Split within the trim size, keep the bleed outside

Booklet exported as spreads

Page 1 single A4, then A3 spreads: halve only the spreads – gives consecutive A4 pages.

```
passermark-cli split broschuere.pdf einzelseiten.pdf --set only_spreads=true
```

Sheet with 2 × 4 cards into single cards

Grid on page 1.

```
passermark-cli split bogen.pdf karten.pdf --set mode=grid --set cols=2 --set rows=4 --pages 1
```

3 · Presets: settings as a file

Anyone who needs the same settings again and again stores them once in a file. The template with all settings and their defaults is provided by `settings`.

```
passermark-cli settings cutcontour > sticker.json
```

Then adapt the file in a text editor – only change the values after the colon:

```
{
  "job": "cutcontour",
  "settings": {
    "shape": "rounded",
    "corner_mm": 4,
    "offset_mm": 2,
    "bleed_mm": 2,
    ...
  }
}
```

The values that differ from the default are enough – Passermark fills in the rest when loading. `tidy` cuts a long file down to exactly these values (workflows too):

```
passermark-cli tidy sticker.json
```

Use it – individual values can still be changed with `--set`:

```
passermark-cli cutcontour bogen.pdf bogen-cut.pdf --preset sticker.json
passermark-cli cutcontour bogen.pdf bogen-cut.pdf --preset sticker.json --set bleed_mm=3
```

A preset always belongs to one job – a CutContour preset cannot be used for `separate`. Values with special characters (e.g. `font_map` with Windows paths) are easiest to enter in the preset instead of with `--set`.

Presets from the program

It is even easier in the program itself: the windows for CutContour, CMYK/crop and the print dialog have a **Preset** line at the top. Set the settings as desired, **Save…** and give a name. Only what differs from the default is saved – to have everything in the file, tick “Save all settings” in the **…** menu. The command line then finds the preset by its name – without a path:

```
passermark-cli presets cutcontour
passermark-cli cutcontour bogen.pdf bogen-cut.pdf --preset "Sticker rund"
```

The presets are stored as JSON files per job – on Linux in `~/.config/passermark/presets/`, on Windows in `%APPDATA%\Passermark\presets\`. In the program: workspace **Automation** → **Presets** opens the folder. Print presets (booklet, poster, step & repeat) belong to the job `impose`.

4 · Whole folders at once

Linux (terminal): cut all PDFs from `eingang`, results to `fertig`, errors are reported.

```
mkdir -p done
for f in input/*.pdf; do
    passermark-cli cutcontour "$f" "done/$(basename "$f" .pdf)-cut.pdf" \
        --preset sticker.json --quiet || echo "ERROR: $f"
done
```

Windows (PowerShell):

```
$cli = "C:\Program Files\Passermark\passermark-cli.exe"
New-Item -ItemType Directory -Force -Path done | Out-Null
Get-ChildItem input\*.pdf | ForEach-Object {
    & $cli cutcontour $_.FullName "done\$($_.BaseName)-cut.pdf" `
        --preset sticker.json --quiet
    if ($LASTEXITCODE -ne 0) { Write-Host "ERROR: $($_.Name)" }
```

```
}
```

5 · Workflows: several steps as JSON

A workflow is a JSON file with several jobs in a row – each step works on the result of the previous one. The output is only written when all steps succeeded. Workflows are written by hand (or generated by an add-on program); the program itself does not assemble workflows.

```
{
  "passermark": "workflow", "version": 1,
  "name": "Visitenkarten SRA3",
  "description": "Ebenen festschreiben, 85 x 55 mm so oft wie möglich auf SRA3",
  "output": "${dir}/fertig/${stem}-sra3.pdf",
  "steps": [
    {"job": "preflight_fix", "settings": {"fix": "flatten_layers"}},
    {"job": "impose", "pages": "1-2",
     "settings": {"sheet": "SRA3", "step_repeat": true, "crop_marks": true, "bleed_mm": 3}}
  ]
}
```

Entry	Meaning
`input`	Input file or list – several files, images and Office documents are first merged into one PDF. Usually given when calling with `--in`
`output`	Result. Missing folders are created. May be left out when a step prints
`steps`	the steps: `job` as above, plus `settings` (only what differs from the default), `preset` (name or file) and `pages`
`${name}`	Variables: `--var name=wert`, plus the built-in `${input}`, `${dir}` (folder of the input), `${stem}` (name without extension), `${date}`, `${workflow_dir}`

Unknown entries and wrong values are errors – a typo never silently falls back to the default. `check` reports all problems at once without running anything.

Check and run

Paths given on the command line are relative to the current folder, paths in the file relative to its folder.

```
passermark-cli check visitenkarten.json --in karten.pdf
passermark-cli run visitenkarten.json --in karten.pdf
```

Own output and variables

Workflow `kunde.json`: shrink, then booklet; the output ends up in a folder per customer.

```
passermark-cli run kunde.json --in katalog.pdf --var kunde=Muster
passermark-cli run kunde.json heft.pdf --in katalog.pdf --var kunde=Muster
```

```
{
  "passermark": "workflow", "version": 1,
  "output": "${dir}/${kunde}/${date}-${stem}.pdf",
  "steps": [
    {"job": "shrink", "settings": {"downsample": true, "target_dpi": 300}},
    {"job": "impose", "settings": {"sheet": "A4", "handling": "booklet", "booklet_fold_marks": true}}
  ]
}
```

Conditions: if – else

A step with `"if"` only runs when the condition holds. `"choose"` takes the first branch whose condition holds, otherwise `"else"`. `fail` stops with a message (the watcher puts the file into `failed/`), `stop` finishes early. What is checked is always the PDF as it looks after the steps before.

```
{
  "passermark": "workflow", "version": 1,
  "output": "${dir}/fertig/${stem}.pdf",
  "steps": [
```

```
      {"job": "shrink", "if": {"size_mb": {">": 20}}},
      {"choose": [
        {"if": {"format": "A5", "pages": {">=": 8}},
         "steps": [{"job": "impose", "settings": {"sheet": "A4", "handling": "booklet"}}]},
        {"if": {"format": "A4"},
         "steps": [{"job": "stop", "message": "A4 – bleibt wie es ist"}]},
        {"else": [{"job": "fail", "message": "Format ${format} wird hier nicht verarbeitet"}]}
      ]}
    ]
  }
```

Fact	Meaning
`pages`, `size_mb`	number of pages, file size in MB
`format`, `page_format`	standard size of the first page (its trim size if it has one) or of the page itself: A4, SRA3 …, `""` = none
`width_mm`, `height_mm`, `orientation`, `mixed_sizes`	size, orientation (`portrait`/`landscape`), different page sizes
`trim`, `bleed_mm`	trim size present, bleed in mm
`errors`, `warnings`, `rgb`, `spot_colors`, `fonts_missing`, `transparency`	results of the document check (only runs when needed)
`filename`, `output_intent`, `layers`	file name, output intent, number of layers
`var.NAME`	variable from `--var` or the watcher's `vars`

Comparisons: a value on its own means "equal", otherwise `{">=": 8}` – possible are `== != > >= < <= in matches` (`matches` = pattern with * and ?). Several entries must all hold, any, `all` and `not` combine. In messages `${format}`, `${pages}` … are filled in.

Checking conditions

`check` shows the branches and reports typos; `run --json` reports every decision with its reason.

```
passermark-cli check verteilen.json --in heft.pdf
```

Workflow folders: call workflows by name

If a workflow is in one of the workflow folders, its name is enough (file name without `.json`). The folders are searched in this order – the first workflow with the name counts. Presets and CSV files the workflow needs simply go next to it.

	own	shared (administrator)	for all users (administrator)
Linux	`~/.config/passermark/workflows/`	any folders, e.g. a network share	`/etc/passermark/workflows/`
Windows	`%APPDATA%\Passermark\workflows\`	e.g. `\\server\print\workflows`	`%ProgramData%\Passermark\workflows\`

The administrator enters the shared folders in the administration (**General → Additional workflow folders**). Who may put workflows there is decided by the folder's permissions – e.g. a group with write access. Nobody needs administrator rights for that.

```
passermark-cli workflows
passermark-cli workflows --dirs
passermark-cli run Visitenkarten --in karten.pdf
```

Create your own folder once yourself (Linux: `mkdir -p ~/.config/passermark/workflows`); `workflows --dirs` shows where it belongs. Up to 1.14 the folders were called `ablaeufe` – an old folder is renamed the first time.

6 · Printing

The `print` step sends the PDF to a printer – the same way as the print dialog: page handling (booklet, step & repeat, poster …), the printer's colour profile from the administration, automatic tray selection. The PDF itself stays unchanged; afterwards the workflow can continue or save.

```
{
  "passermark": "workflow", "version": 1,
  "steps": [
    {"job": "print", "settings": {
      "printer": "Canon_iR-ADV_C3822i", "paper": "A4", "tray": "Fach 2",
      "duplex": "long", "copies": 2, "finishing": {"staple": "links oben"},
      "dry_run": true}}
  ]
}
```

Setting	Meaning
`printer`	Name as shown by `passermark-cli printers`; empty = default printer
`paper`, `tray`, `media`, `output_bin`	Paper size, tray, media, output bin – driver value or the displayed name, a unique part is enough (`"Fach 2"`)
`"paper": "auto"`	Paper matching the PDF: with size the page, with a booklet two pages side by side (A5 pages → A4). Without `paper` the printer uses its default paper
`duplex`, `color`	`none` / `long` / `short` or `color` / `gray` – or the driver value
`finishing`	Finishers: `staple`, `punch`, `fold`, `saddle`, `trim`, `stacker`
`finisher_preset`	Finisher template from the administration – on Windows the way to all vendor functions (original driver dialog)
`options`	further raw driver options: `{"Schlüsselwort": "Wert"}`
`copies`, `collate`, `reverse`	Copies, collated, reverse order
`layout`	Page handling as in the print dialog, e.g. `{"handling": "booklet"}`
`dry_run`	`true` = send nothing, only show what would be sent

Printers and what they can do

All options, trays, sizes and finishers of a printer – what is listed there goes into the workflow.

```
passermark-cli printers
passermark-cli caps "Canon_iR-ADV_C3822i"
```

A value that matches no choice is an error that lists all possible values. What the driver cannot do with the installed hardware (e.g. punching without a punch unit) is rejected. Printer names with spaces go in quotation marks; on Linux the CUPS queue name has no spaces (`Canon_iR-ADV_C3822i`).

Step & repeat with step 2: printing works like saving – the finished step-2 sheets, with the step-2 target size as the paper.

Print log

When the print log is on (**File → Settings**, or by the administrator for everybody), every print job is stored as JSON with all its settings – plus a list `printlog.csv`. Folder: `~/.local/state/passermark/printlog` or `%LOCALAPPDATA%\Passermark\printlog`, or the one set by the administrator. Print exactly the same again: `passermark-cli reprint protokoll.json` (`--dry-run` to test, `--in` for another file).

Colour: papers, soft proof, colour check

Everything about colour is in the program's **Colour** menu (and in the Prepress workspace). **Papers and measurements**: your own papers without admin rights – name, weight, surface, measured paper white (instrument via ArgyllCMS, measurement file or by hand) and the fitting profile per printer; suggestions by paper white (ΔE), surface and printer. Papers for everybody are

created by the admin under **Administration → Default settings → Materials**. **Get standard profiles** loads PSO Coated v3, PSO Uncoated v3, FOGRA39 … free from registry.color.org. **Colour check** (`Ctrl+Shift+F`) checks the document against the printing condition (RGB, output intent, ink coverage, spot colours, paper white, fonts) and suggests what to do – with a button to run it. **Soft proof** (`Ctrl+Y`) shows the page in the print colours. **Output for print shop** corresponds to the job `press`. Measuring and suggestions exist only in the program; printing from the command line uses the printer's profile from the administration.

Plan printing on rolls

In the program: workspace **Plan & Poster → Plan printing**. Every page is a plan. If the plan has two outer lines, printing goes up to the outer one – it disappears and an exact margin stays around the plan. Otherwise the content plus 5 mm is printed. Each plan goes onto the roll with the shortest length (rotation allowed – an A1 goes across the 841 roll instead of upright on the 594 roll), with equal length onto the narrower one. Only rolls with plan paper count for plans (default 75/80 g/m², `weights`). Run length: only the plan sizes, per roll for one set and for all copies. Default is uncollated (plan 1 ×3, plan 2 ×3 …), `collate` prints set by set, `"order": "roll"` first all plans of the narrowest roll.

Passermark reads the rolls from the device (IPP, otherwise SNMP; address from the administration or the queue). If the device reports nothing, the rolls entered in the program under **Rolls …** apply. `--rolls 594,841,914` sets them directly – so `--dry-run` computes the run length even without a printer. Every job is logged with roll and run length in the print log (`roll_mm`, `run_m`).

Posters and banners: workspace **Plan & Poster → Poster printing** or `--kind poster`. Quality print, pieces side by side with the least paper; whatever is wider than every roll is cropped or split into strips (`overwidth`). Banners: hem/pocket (`hem_mm`), grommet marks (`grommet_spacing_mm`), `cut_contour` draws the cut line in the spot colour CutContour for print & cut, `--plt` writes a cut file. Print materials (category plan/quality print, grammage, media type, quality, ink amount, profile) are created in the program and selected with `material`. Devices without a driver (Roland via VersaWorks) get the PDFs in an output folder: `"printer": "folder:Name"`.

Run length for a set of plans

Three copies, only compute – nothing is sent.

```
passermark-cli plan Mappe.pdf --rolls 594,841,914 --copies 3 --dry-run
```

Printing plans in a workflow

Step "plan" with the same settings as in the program (all of them: passermark-cli schema plan).

```
{"job": "plan", "settings": {"printer": "ColorWave", "copies": 2, "color": "gray",
                             "order": "roll", "fix_hairlines": true}}
```

7 · Examining documents

`inspect` shows pages, sizes, trim size and bleed per page, document information and with `--check` the check results (fonts, layers, images, colours) – the same as **File → Properties** in the program.

```
passermark-cli inspect karten.pdf --check
```

8 · For other programs

Other programs (scripts, watcher, shop and intranet connections) use Passermark only through the command line – with `--json` everything comes as JSON lines with fixed error codes (`usage`, `not_found`, `invalid`, `failed`, `cancelled`). The full description is in `docs/cli-api.md`.

```
passermark-cli version --json
passermark-cli schema impose
```

```
passermark-cli run visitenkarten.json --in karten.pdf --json
```

9 · Drop folders: watcher

With `serve` Passermark watches folders: every file you put in (also over the network or from a scanner) runs through the workflow assigned to the folder – it is processed, printed or both. Which folder has which workflow is set in `watch.json`:

```
{
  "passermark": "watch", "version": 1,
  "interval": 5, "settle": 5,
  "folders": [
    {"in": "eingang/visitenkarten", "workflow": "visitenkarten.json"},
    {"in": "eingang/drucken", "workflow": "drucken-a4.json", "done": "",
     "patterns": ["*.pdf"], "name": "A4 heften"}
  ]
}
```

Entry	Meaning
`in`	the drop folder (must exist)
`workflow`	name from the workflow folders or JSON file
`out`	results, default `output`; `""` = keep no result. A workflow that only prints keeps none without `out`
`done`	originals afterwards, default `processed`; `""` = delete
`error`	originals that failed, with a `.log` next to them giving the reason; default `failed`
`patterns`	which files, default `["*.pdf"]`
`vars`	variables for the workflow, like `--var`
`interval`, `settle`	seconds between two passes; how long a file must stay unchanged before it counts as completely copied

Relative paths: `in` and `workflow` from the folder of `watch.json`, `out`/`done`/`error` from the drop folder. The result is named like the file and never overwritten (`-1`, `-2` …). Half-copied files (`.part`, `.tmp`) and hidden files are never touched.

Check, run once, keep running

Without `--config` your own `watch.json` applies (`~/.config/passermark/`, `%APPDATA%\Passermark\`), otherwise the one for everybody (`/etc/passermark/`, `%ProgramData%\Passermark\`). Stop with **Ctrl+C** – a file being processed goes back into its folder.

```
passermark-cli serve --config watch.json --check
passermark-cli serve --config watch.json --once
passermark-cli serve --config watch.json
```

`passermark-cli serve --init` creates a ready template – with an example workflow and example folder `hotfolder/beispiel`; `--system` for all users. An existing `watch.json` is never overwritten.

As a service without a logged-in user

Best with a user of its own without special rights that may write to the drop folders. Linux: `linux/passermark-watch.service` (systemd, the steps are in the file), log with `journalctl -u passermark-watch -f`. Windows: `windows/watch-task.ps1` creates a task at system start, log in `%ProgramData%\Passermark Watcher\` – the printers must be set up for this user.

Several watchers may watch the same folder: each file is first moved into `.passermark` and processed only once. After a crash, files left behind come back at the next start.

10 · Tips and errors

Message / problem	Solution
No motif found	Lower the tolerance (`--set detect.tolerance=15`) or set the mode (`--set detect.mode=color`)
Spots are detected as motifs	Raise the tolerance, increase `detect.min_size_mm`
Parts of a logo get separate contours	Increase `detect.gap_mm`, e.g. 3
Takes a long time	Keep `dpi` at 200; 300 easily takes twice as long. Passermark automatically uses several CPU cores for large pages with many objects
The computer should stay free for other work	Limit the cores: Linux `PASSERMARK_WORKERS=2 passermark-cli …`, Windows (PowerShell) put `$env:PASSERMARK_WORKERS=2` in front; 1 = no parallelism
Preset is for "…"	The preset belongs to a different job
Value is read as text instead of a number	Use a decimal point: `bleed_mm=2.5` (not 2,5)
Spaces in the file name	Put names in quotation marks: `"mein bogen.pdf"`
"… is not a job" with a .json	Start workflows with `run`: `passermark-cli run ablauf.json --in datei.pdf`
"is a workflow, not a preset"	The file has `steps` – run it with `run`, do not load it with `--preset`
Workflow not found	`passermark-cli workflows` shows all names, `--dirs` the folders
Print comes out on A4 instead of A3	Without `paper` the printer uses its default paper – give `"paper": "A3"` or `"auto"`
Watcher does not pick up a file	check the patterns in `patterns`; the file is only taken once it has stayed unchanged for `settle` seconds
"The result would end up in the input folder again"	set `out` to a folder of its own (not `""`)